

Vorlesung Cybersicherheit, Sommersemester 2025

Reverse-Engineering und Malware Analyse

MSc. David Paaßen

Prof. Dr.-Ing. Lucas Vincenzo Davi

Fakultät für Informatik

Arbeitsgruppe Systemsicherheit <https://www.syssec.informatik.uni-due.de/>

Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi

What is Reverse-Engineering?

compilation

```
1  #include<stdio.h>
2
3  int main() {
4      printf("Hello World\n");
5      return 0;
6  }
7
```

```
004034F0  0C 00 00 FF D0 BE B0 82 40 00 56 E8 52 32 00 00  ...ÿD%,@.VèR2..
00403500  56 FF 15 50 81 40 00 8D 74 06 01 80 3E 00 75 EA  Vy.P.@..t..€>.uê
00403510  6A 0A E8 AB 32 00 00 6A 08 E8 A4 32 00 00 6A 06  j.è«2..j.è²2..j.
00403520  A3 44 A2 42 00 E8 98 32 00 00 3B C3 74 0F 6A 1E  fD¢B.è~2..;Ãt.j.
00403530  FF D0 85 C0 74 07 80 0D 4F A2 42 00 40 55 FF 15  ÿD...Àt.€.0¢B.@Uÿ.
00403540  44 80 40 00 53 FF 15 A0 82 40 00 A3 18 A3 42 00  D€@.Sÿ.., @.£.£B.
00403550  53 8D 44 24 34 68 B4 02 00 00 50 53 68 E8 16 42  S.D$4h'...PShè.B
```

What is Reverse-Engineering?

```
004034F0 0C 00 00 FF D0 BE B0 82 40 00 56 E8 52 32 00 00 ...ÿðX°,@.VèR2..  
00403500 56 FF 15 50 81 40 00 8D 74 06 01 80 3E 00 75 EA Vy.P.@..t..€>.uê  
00403510 6A 0A E8 AB 32 00 00 6A 08 E8 A4 32 00 00 6A 06 j.è«2..j.è¤2..j.  
00403520 A3 44 A2 42 00 E8 98 32 00 00 3B C3 74 0F 6A 1E ED¢B.è~2..;Ät.j.  
00403530 FF D0 85 C0 74 07 80 0D 4F A2 42 00 40 55 FF 15 ÿÐ...Ät.€.O¢B.@Uÿ.  
00403540 44 80 40 00 53 FF 15 A0 82 40 00 A3 18 A3 42 00 D€@.Sÿ..,@.£.£B.  
00403550 53 8D 44 24 34 68 B4 02 00 00 50 53 68 E8 16 42 S.D$4h'...PShè.B
```



Dynamic
Analysis



Static
Analysis



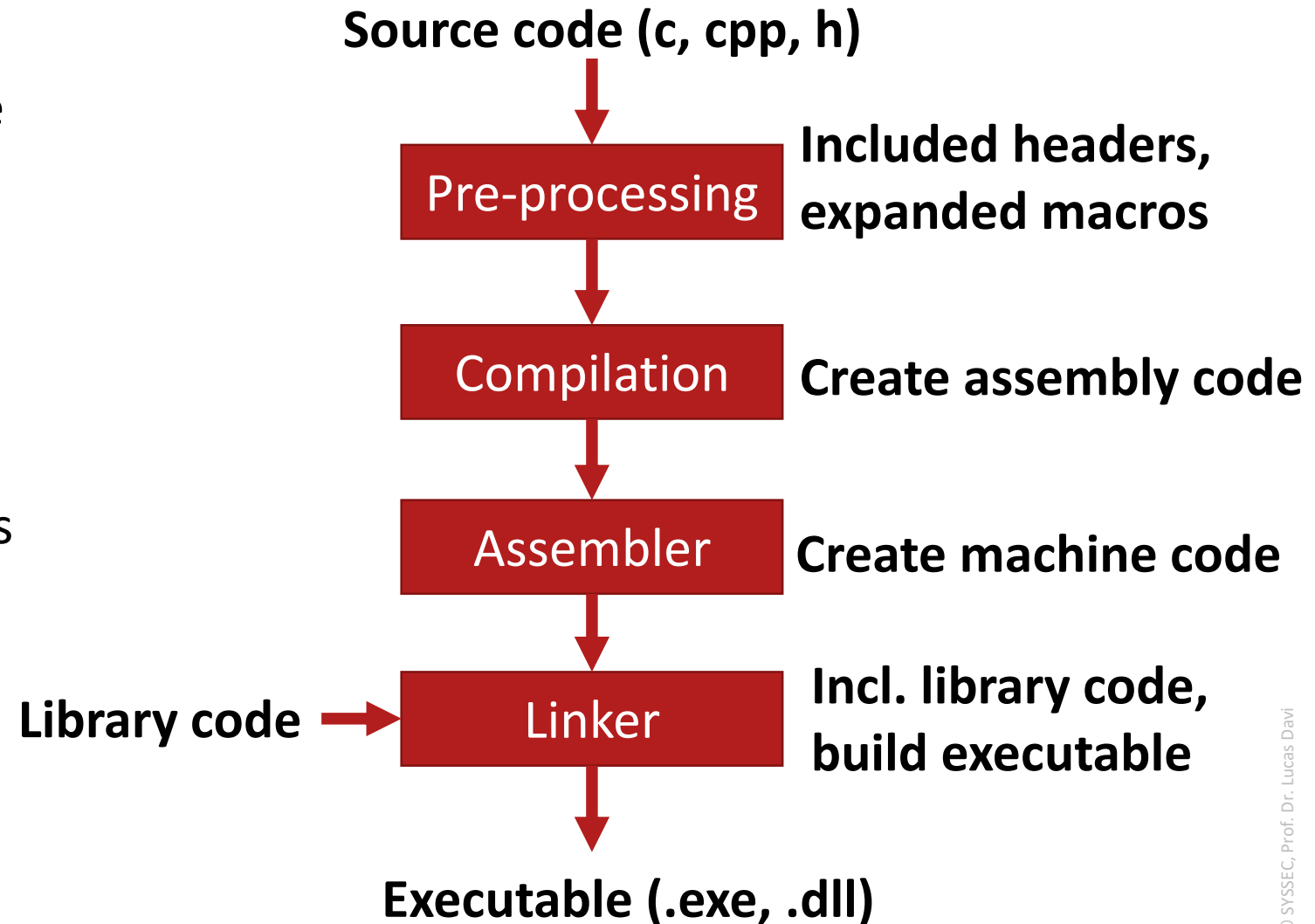
Applications of Reverse-Engineering

- Verify program's functionality
- Analyze software to detect bugs
 - Security vulnerabilities
- Patch programs without source code
- Detect malicious software

Why is reverse-engineering challenging?

Creation of an Executable (C/C++)

- From source to an executable
- Pre-processing
- Compilation
- Assembler
- Linker
- Executable no longer contains any source code info
 - Variable types/names
 - Function names
 - Comments



Creation of an Executable Example

Source code (hello.c):

```
#include<stdio.h>
```

```
int main() {  
    printf("Hello World\n"); //Output "Hello World"  
    return 0;  
}
```

Creation of an Executable Example

After pre-processing:

```
#1 "hello.c"
#1 "<built-in>"
#1 "<command-line>"
#1 "hello.c"
[...] <content of stdio.h> [...]
#2 "hello.c" 2
#3 "hello.c",
int main() {
    printf("Hello World\n");
    return 0;
}
```

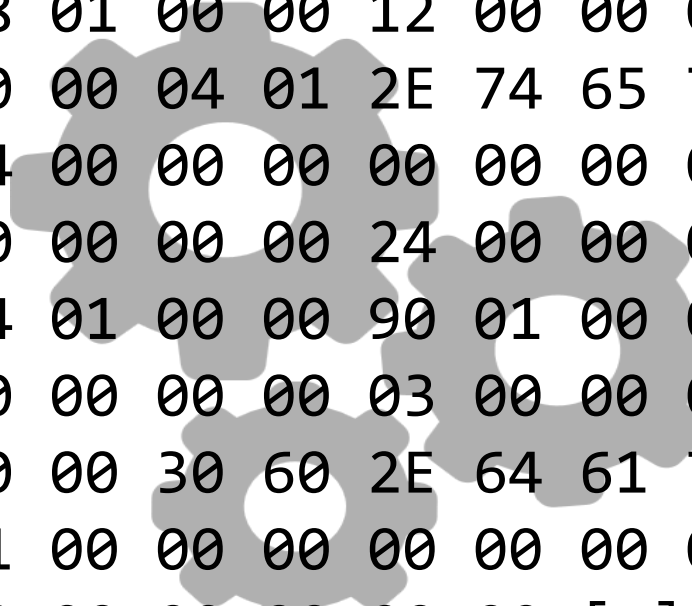
Creation of an Executable Example

Assembly:

```
[...]  
pushl %ebp  
.cfi_def_cfa_offset 8  
.cfi_offset 5, -8  
movl %esp, %ebp  
.cfi_def_cfa_register 5  
andl $-16, %esp  
subl $16, %esp  
call ____main  
movl $LC0, (%esp)  
call _puts  
[...]
```

Creation of an Executable Example

- Object code
- No longer human readable



```
hello.o
4C 01 06 00 00 00 00 00
B8 01 00 00 12 00 00 00
00 00 04 01 2E 74 65 78
74 00 00 00 00 00 00 00
00 00 00 00 24 00 00 00
04 01 00 00 90 01 00 00
00 00 00 00 03 00 00 00
20 00 30 60 2E 64 61 74
61 00 00 00 00 00 00 00
00 00 00 00 00 00 [...]
```

Creation of an Executable Example

- Final executable program
- Includes header, compiled code, libraries, data
- Variable/function names or comments only as debug info

hello.exe

```
4D 5A 90 00 03 00 00 00
04 00 00 00 FF FF 00 00
B8 00 00 00 00 00 00 00
40 00 00 00 00 00 00 00
00 00 00 00 80 00 00 00
0E 1F BA 0E 00 B4 09 CD
21 B8 01 4C CD 21 54 68
69 73 20 70 72 6F 67 72
61 6D 20 63 61 6E 6E 6F
74 20 62 65 20 72 [...]
```

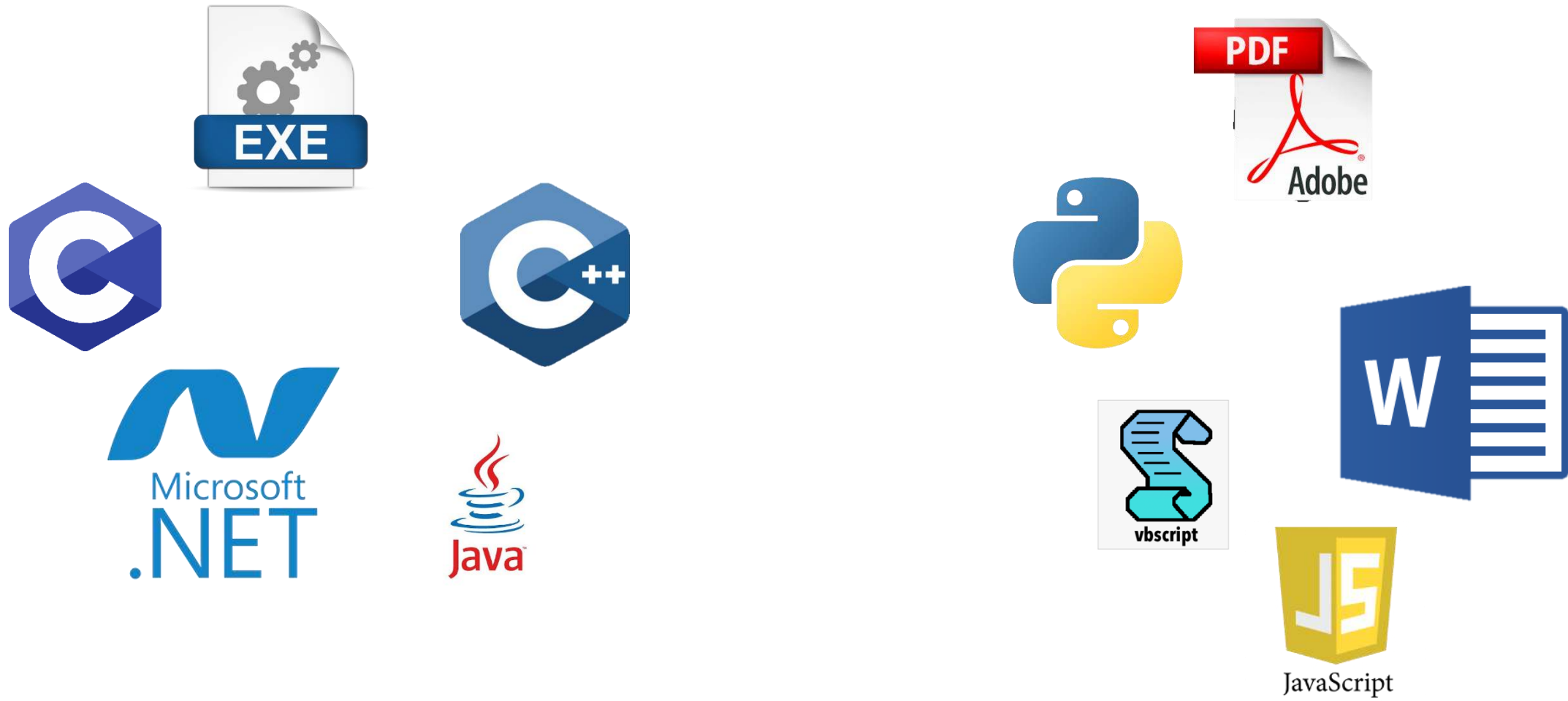
What is Malware?

- Malware stands for “malicious software”

“A program that is inserted into a system, usually covertly, with the intent of compromising the confidentiality, integrity, or availability of the victim’s data, applications, or operating system or of otherwise annoying or disrupting the victim.” – NIST

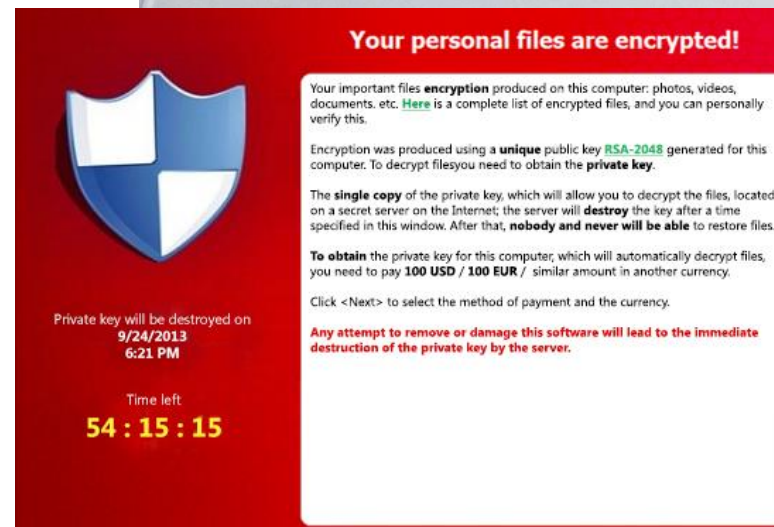
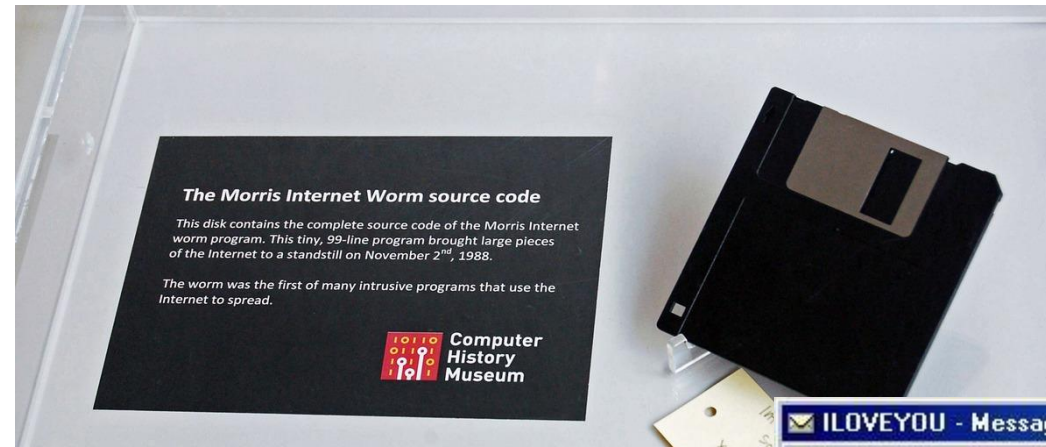
- Predominantly written for Windows systems

Forms of Malware



History of Malware

- 1988: Morris worm
- 2000: ILOVEYOU
- 2009: Zeus
- 2013: Cryptolocker
- 2016: Mirai
- 2022: Costa Rica Ransomware attacks



Goals of Malware Analysis

- Propagation and infection vector
 - How does the malware spread?
- Persistency
- Functionality and malware type
- Removal
- Bypass anti-analysis techniques

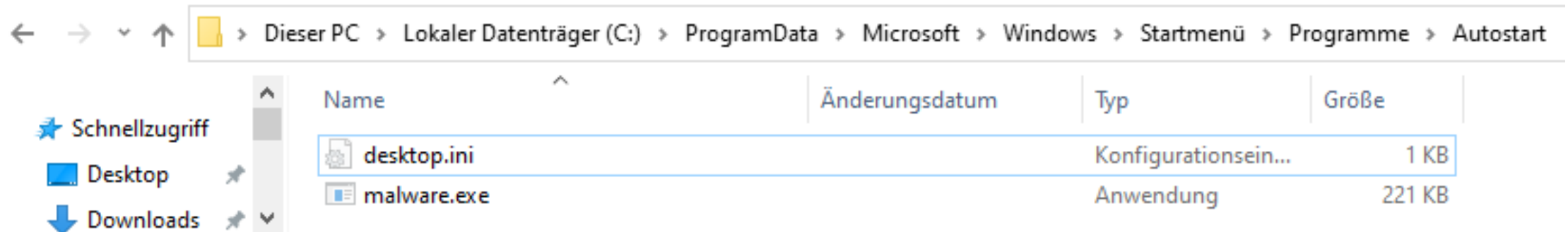
- (Infected) program downloads
- E-mail attachments
- Exploits
 - Browsers (e.g., Malvertising)
 - Adobe PDF Reader, Microsoft Word
 - Operating system

- Mechanism to execute the malware
 - Run malicious code after a system restart
- Popular methods
 - Windows Startup Folder
 - Registry Run Keys
 - Services

Persistence: Windows Startup Folder

Simply copy malware binary into special directories

- `C:\Documents and Settings\All Users\Start Menu\Programs\Startup`
- `C:\Users\<Username>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup`
- `C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`



Persistence: Registry Run Keys

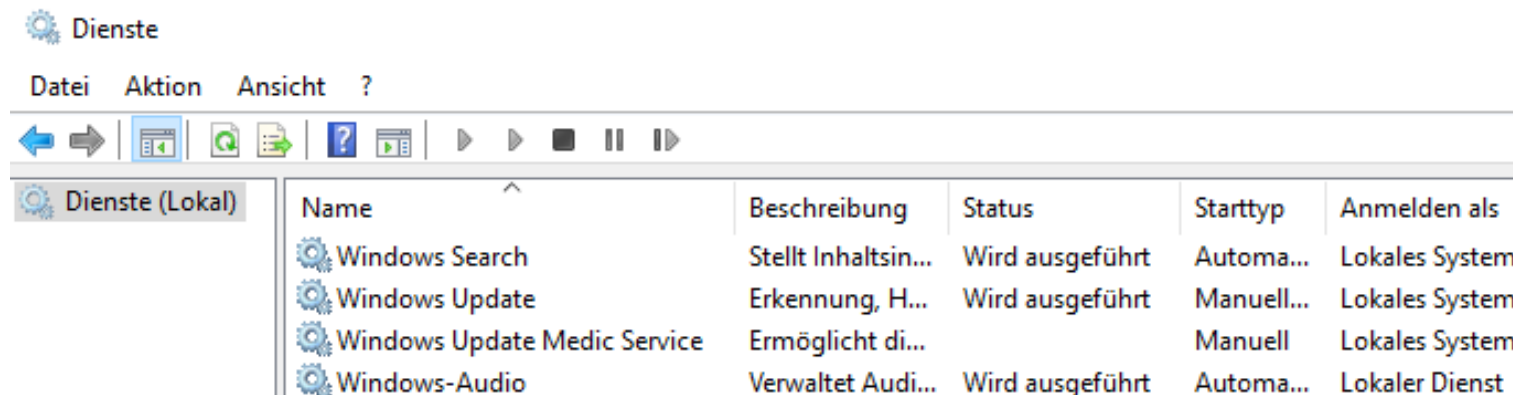
- Plethora of different keys/entries
- Similar to the startup folder technique
- Most popular example
 - HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run
 - Key has the binary path as a value

```
Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run]
"SecurityHealth"=hex(2):25,00,77,00,69,00,6e,00,64,00,69,00,72,00,25,00,5c,00,\
  73,00,79,00,73,00,74,00,65,00,6d,00,33,00,32,00,5c,00,53,00,65,00,63,00,75,\
  00,72,00,69,00,74,00,79,00,48,00,65,00,61,00,6c,00,74,00,68,00,53,00,79,00,\
  73,00,74,00,72,00,61,00,79,00,2e,00,65,00,78,00,65,00,00,00
"VBoxTray"="C:\\Windows\\system32\\VBoxTray.exe"
```

Persistence: Services

- Services are background processes which are automatically executed
- Install via Microsoft utility: installutil malware.exe
- Install via PowerShell: New-Service -Name "Harmless" -BinaryPathName malware.exe
- Install Service Control Manager API: CreateService/StartService
- Manually via registry entry



- Trojan
- Worm
- Virus
- Cryptominer
- Backdoor
- Bot
- Adware/Ad-clicker
- Ransomware

```
program virus :=
{1234567;
  subroutine infect-executable :=
    {loop: file = random-executable;
     if first-line-of-file = 1234567
       then goto loop;
     prepend virus to file;
    }
  subroutine do-damage :=
    {whatever damage is desired}
  subroutine trigger-pulled :=
    {return true on desired conditions}
  main-program :=
    {infect-executable;
     if trigger-pulled then do-damage;
     goto next;
    }
  next: }
```

Fig. 1. Simple virus 'V'.

Functionality: Ransomware

- Encrypt files on hard drive, network shares, external drives, cloud storage, backup systems
- Demand ransom to decrypt files
 - Usually via crypto currency
- Do not need to be stealth
- Does not require persistency
- Commonly used in targeted attacks

We have hacked your network, downloaded and encrypted your data.
You can recover your data and prevent data leakage to public.
Please upload your note **RECOVER-FILES.txt** using the form below and start recovering your data.
After you upload note, you will be provided with further instructions.

- Search for files: FindFirstFile/FindNextFile
- Filter list: “*.exe”, “*.dll”, “*.ini”, “C:\Windows\”
- Encryption: CryptEncrypt, AES
- Delete backups: “vssadmin.exe delete shadows /all /quiet”
- Write ransom note
 - Strings: “Note.txt”, “FILE_RECOVERY.txt”, “PAY X BTC TO”
 - CreateFile/WriteFile

Functionality: Ransomware Common Mistakes

- Use custom made or weak encryption routines
 - Exploit cipher weaknesses to break encryption
- Use keys that are not random
 - Static keys
 - Keys based on known information
 - User name, time stamp, MAC address
- Insufficient key length

- Find all malware components
 - Start with persistency method
 - Find and analyze related files
 - Execute malware and monitor file system
- Remove malware
 - Disinfect files
 - Read-only, system, hidden files, alternate data stream
 - Fake timestamps

How to impede reverse-engineering?

- Analyze a program without runtime information
- Strings
 - Human readable information
 - No complex analysis required
- Disassembly
- CFG

4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....ÿÿ..
B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	,.....@.....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00 00 00 00 00 00 00 00 00 00 00 00 00 80 00 00	€...
0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°..'!.,.LÍ!Th
69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....\$.

```

push    ebp
mov     edx, 104h
mov     ebp, esp
push    ebx
sub     esp, 14h
mov     ebx, [ebp+lpFilename]
mov     [esp+8], edx    ; nSize
mov     eax, ds:hModule
mov     [esp+4], ebx    ; lpFilename
mov     [esp], eax     ; hModule
call    GetModuleFileNameA
sub     esp, 0Ch
test    eax, eax
mov     edx, 0FFFFFFFh
jz      short loc_402CB5
    
```

```

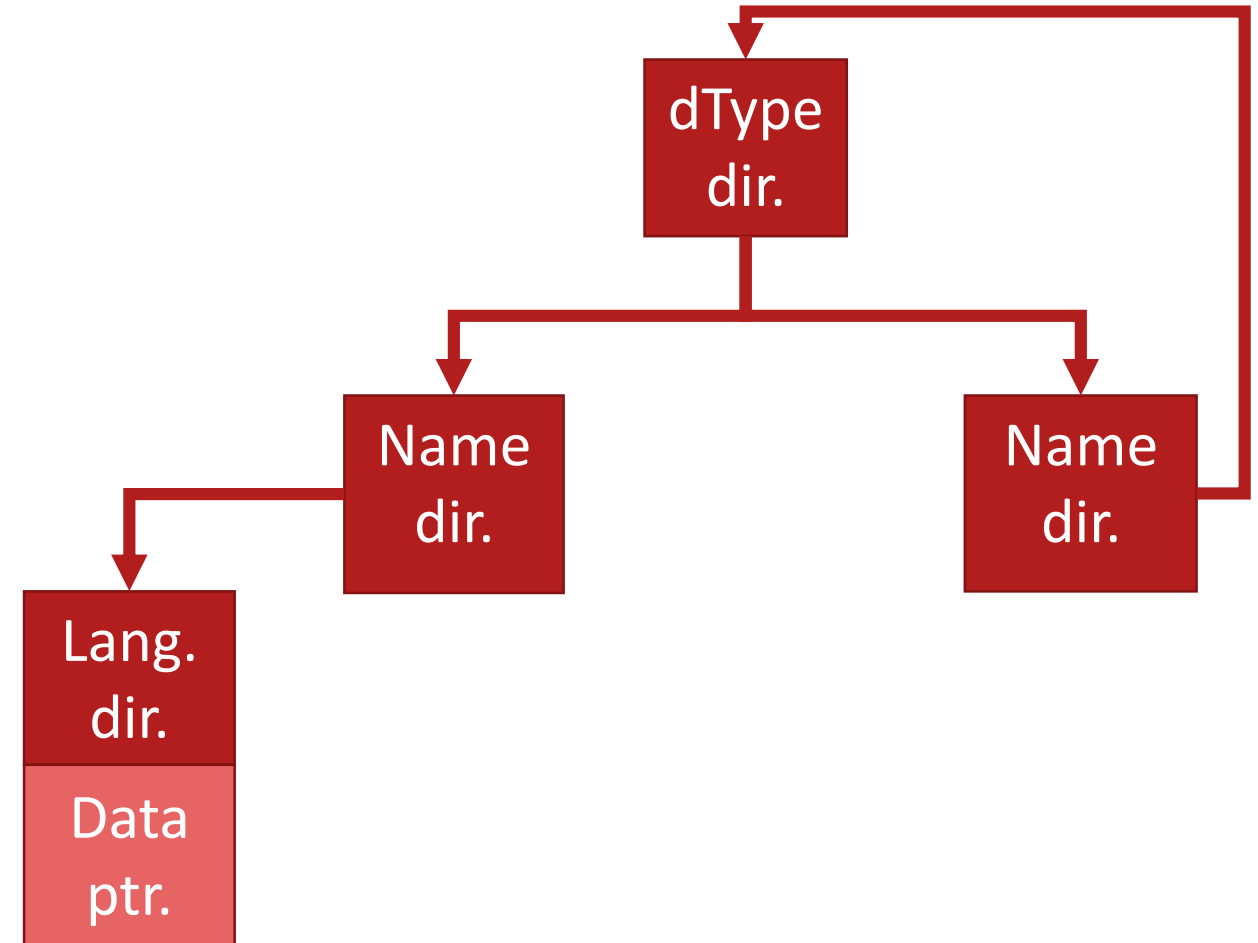
mov     [esp], ebx    ; Str
mov     ecx, 5Ch ; '\
mov     [esp+4], ecx  ; Ch
call    strchr
mov     edx, eax
sub     edx, ebx
    
```

```

loc_402CB5:
mov     ebx, [ebp+var_4]
mov     eax, edx
leave
retn
sub_402C70 endp
    
```

Impede Static Analysis

- Prevent static analysis of a program
- Specifically create code that leads to errors or faulty analysis
 - Manipulate resource table
 - Add irrelevant code
 - Obfuscate (parts of) the binary
 - Prevent (correct) disassembly



Garbage Code

- Add useless instructions
 - NOP equivalents
 - Irrelevant code
- “Needle in the haystack”

add **rax**, 1234h

sub **rax**, 1234h

xor **rbx**, 12BCDh

xor **rbx**, 12BCDh

ror **rcx**, 9

rol **rcx**, 9

```
loc_4054C1:
cmp     eax, edx
inc     esi
cmp     ch, ch
mov     edx, [edi]
test    cx, ax
push    esi
cmp     ebx, ebx
xor     edx, [esp+14h+var_14]
cmp     ch, ah
pop     esi
test    esi, 54112F03h
cmp     edx, ecx
jnz     short loc_4054C1
```

```
loc_4054E8:
test    dl, 9Eh
dec     ecx
test    cx, ax
dec     ecx
cmp     cx, dx
dec     ecx
test    cl, dl
dec     ecx
test    ax, cx
push    dword ptr [edi+ecx]
cmp     ecx, ecx
test    eax, ecx
test    cx, cx
test    edx, eax
cmp     eax, edx
cmp     ch, ch
test    cx, ax
xor     [esp+14h+var_14], esi
cmp     ebx, ebx
cmp     ch, ah
test    esi, 54129686h
cmp     cl, cl
test    bx, bx
test    dl, 0C0h
test    cx, ax
cmp     edx, ecx
cmp     eax, 541316A2h
cmp     eax, edx
pop     dword ptr [eax+ecx]
cmp     ebx, ecx
test    bh, ch
test    bl, al
cmp     ecx, 0
jg      short loc_4054E8
```

String Obfuscation

- Recap: Strings can reveal valuable information about a program
 - Error messages
 - Format strings
 - Paths
 - Domains
- Strings obfuscation makes strings unreadable
- Applications like strings.exe become useless
- Deobfuscation at run time (before use)



String Obfuscation Example

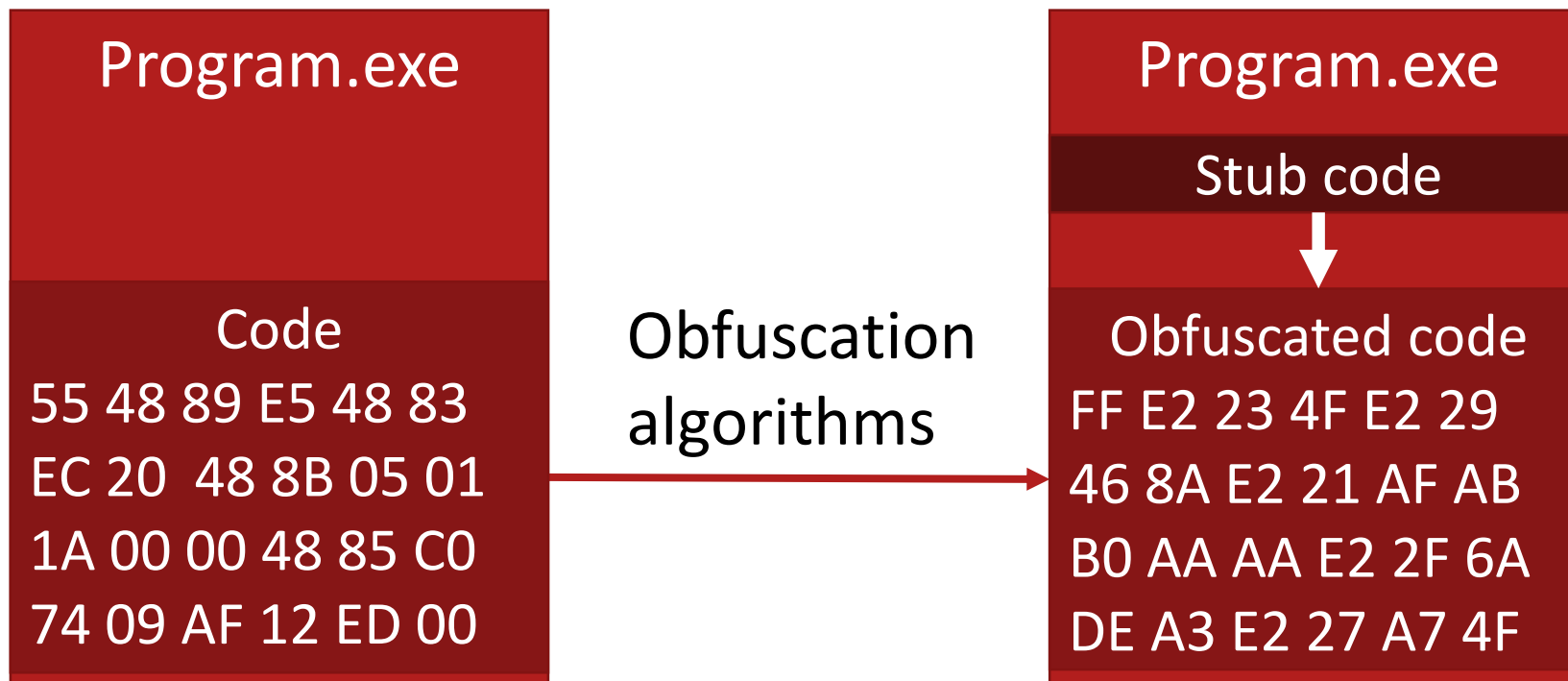
```
void some_function(char s[])  
{[...]}  
  
int main(void) {  
    char a_string[] = "Test";  
    some_function(a_string);  
}  
  
char *deobfuscation(int s[], int len) {  
    [...]  
    return clear_text_string;  
}  
  
void some_function(char s[]){[...]}  
  
int main(void) {  
    int a_string[] = {0xA1,0xCE,0x97,0xC2,0x0A};  
    some_function(deobfuscation(a_string, 5));  
}
```



Note: the obfuscation does not need to be “secure”

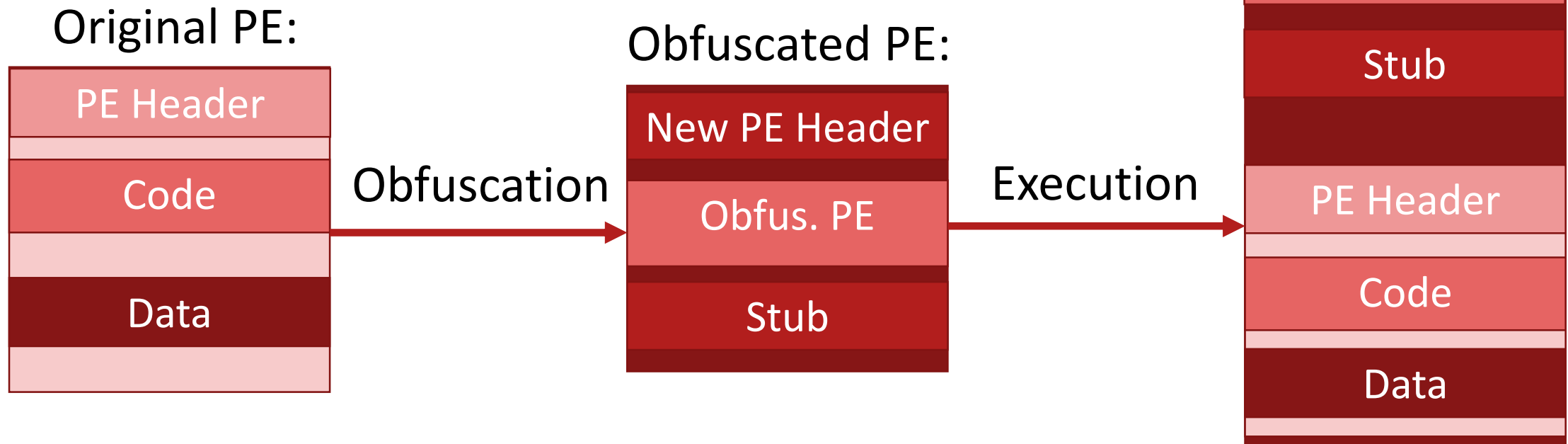
Code Obfuscation

- Obfuscate (part of) the code
- Prevents static analysis of the obfuscated code
- Stub: deobfuscate code at run time in memory and jump to it



PE Obfuscation

- Obfuscate the compiled binary completely
 - Hides code, data, strings, imports, header info
- No useful static analysis possible



- Run (part of) the programs code
- Analyze and observe behavior
 - Executed code, called library functions, file system access, registry access, internet connections
- Dynamic analysis tools
 - Debugger
 - Monitors
 - Sandboxes/virtual machines

Debugger

- Controlled execution
 - Analyzed program is started via a debugger
- Utilize run-time information
 - Dump memory, register/stack values, dynamic jumps, pointers
- Manipulate data
 - Register values, conditions (flags), function arguments, memory/data

The screenshot shows a debugger window for the process 4_2017_default_test_x64_cdecl.exe. The main pane displays assembly code with comments and hex values. The right pane shows the register window with values for RAX, RBX, RCX, RDX, RBP, RSP, RSI, RDI, R8, R9, R10, R11, R12, R13, R14, R15, and RIP. The bottom pane shows a memory dump with addresses and hex values. The status bar at the bottom indicates the current instruction is at address 0000000140001384.

Impede Dynamic Analysis

- Detect dynamic analysis tool
 - Debuggers, monitors
- Detect analysis environment
 - Virtual machines or sandboxes
- Prevent debugger attaching

Detect a Debugger

- Windows API: IsDebuggerPresent
- Malware authors may write custom implementations
- Bypass: set flag to 0

```
call IsDebuggerPresent  
test eax, eax  
jne debugger_detected
```

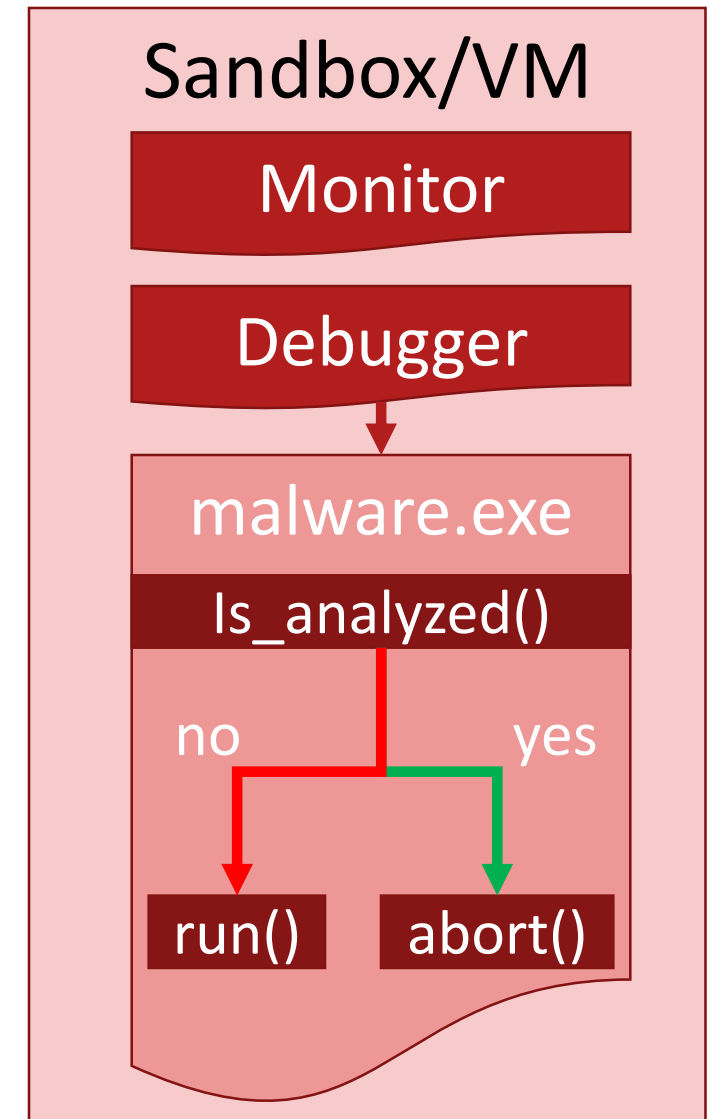
```
mov eax, fs:[0x30]  
mov eax, byte [eax+2]  
test eax, eax  
jne debugger_detected
```

Detect Breakpoints

- Debuggers replace instruction bytes with 0xCC (INT 3)
- Programs can search in their own memory for 0xCC bytes
 - If (code_byte == 0xCC) break_point_detected();
 - Calculate hash/checksum: detects all code manipulations
- Hardware breakpoints
 - Debug registers DR0-DR03 hold hardware breakpoint addresses
 - Check at runtime with GetThreadContext

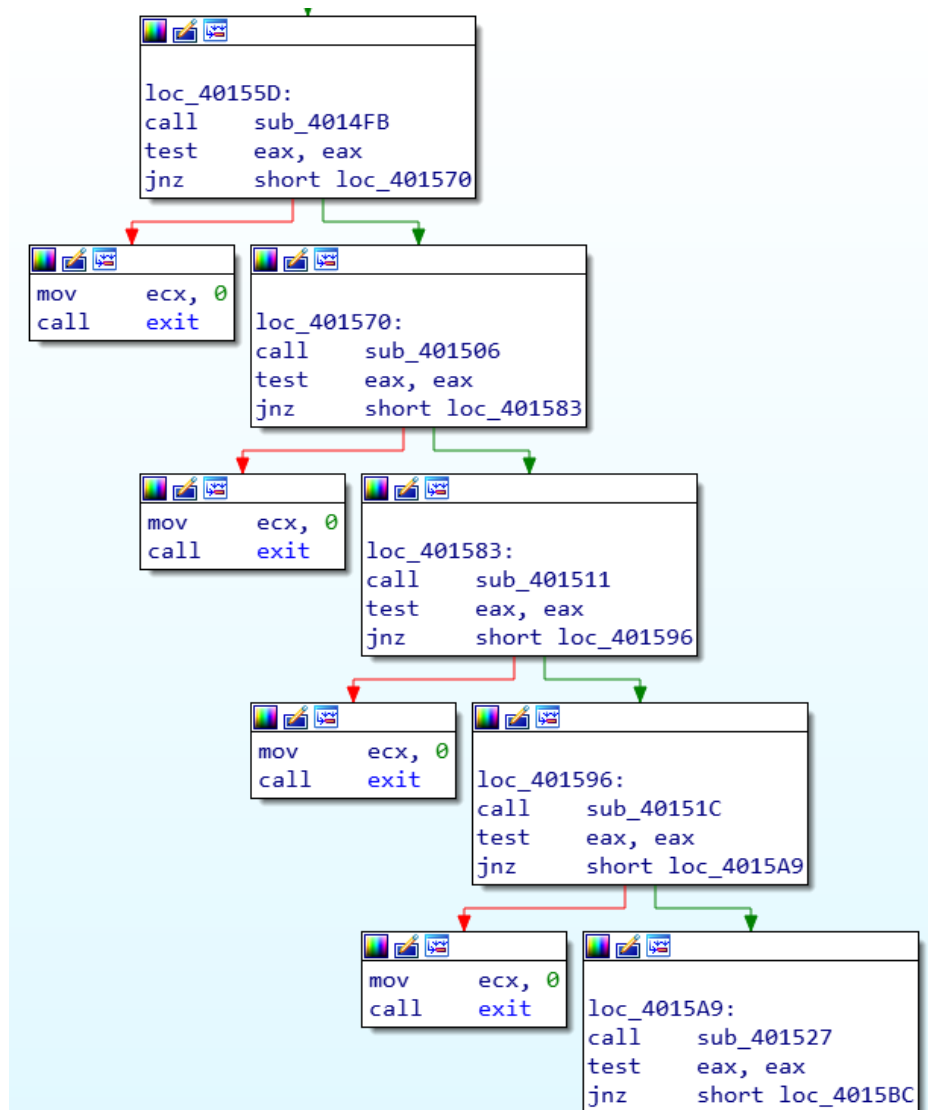
Detect Sandboxes/VMs

- Program analysis is commonly done inside a VM
 - Protect host system, memory snapshots
- Programs check if they run inside a VM
- Artifact based detection
 - Detect differences between VMs/sandboxes and bare-metal machines
- Timing based detection
 - Detect difference in code execution speed



Dynamic Analysis Detection: Common Mistakes

- Implementing checks without additional obfuscation
- Each check returns 0 if test fails and exit() is called
- We can simply patch all checks
- We do not need to analyze any anti-analysis code



Vorlesung Cybersicherheit, Sommersemester 2025

Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi
Fakultät für Informatik
Arbeitsgruppe Systemsicherheit
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi